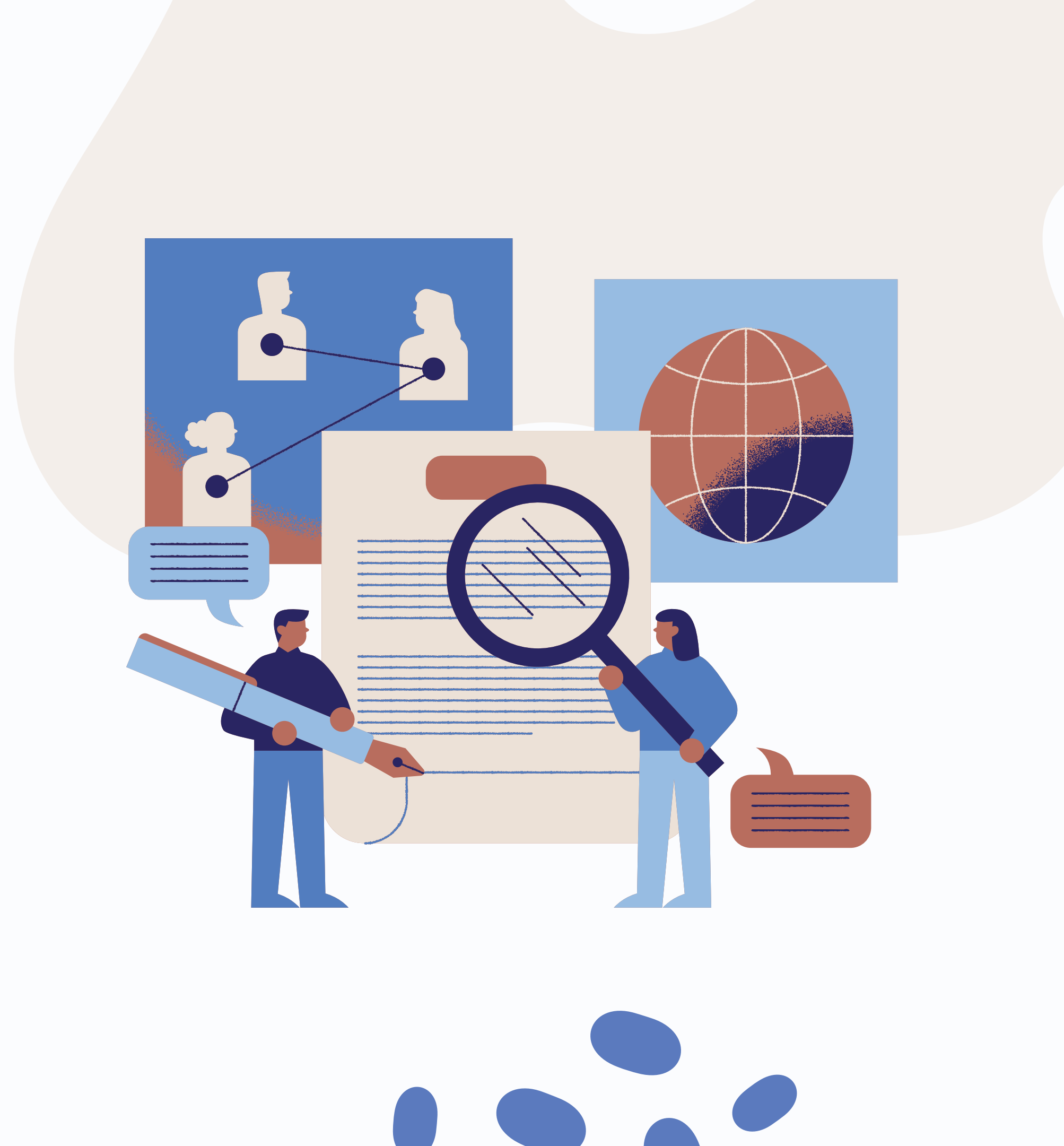


Regression Analysis:

-Nicolas Devin -



DESCRIPTION

This project develops a system that can predict an individual's estimated annual healthcare costs based on data such as age, health condition, lifestyle, and type of insurance coverage. The dataset used includes 100,000 patient records.

Objectives

01

Analyze the key factors that influence patients' annual medical costs through comprehensive exploratory data analysis

02

Build and compare the performance of 11 regression models, ranging from linear models to ensemble and boosting methods, to identify the best-performing model.

03

Evaluate the prediction accuracy of each model using multiple performance metrics

04

Create new feature combinations to help predictive models capture hidden patterns in the data more effectively.

05

Deploy the best-performing model into a simple web application, allowing users to input their information and receive an estimated medical cost prediction

Data Profiling & Quality Check

01

Data Dimension: The dataset is large, comprising **100,000 observations** and **54 features**.

02

Data type exploration: The dataset contains a combination of numerical data types (float64 and int64) and 10 categorical columns (object), such as sex, region, smoker, and education.

03

Missing Value: 30% of data in alcohol_freq column, and the other columns are clean

04

The target variable (**annual_medical_cost**) follows a right-skewed distribution and contains natural outliers. These outliers correspond to valid observations of patients incurring exceptionally high medical costs and are therefore retained to preserve important information for the modeling process.

Data Dimension Summary	
Total Rows	100000
Total Columns	54

Missing Values Table

Nama Kolom	Total Missing	Pecentage
alcohol_freq	30083	30,08

Data Cleaning & Leakage Protection

01

Drop Irrelevant Features:

Removing person_id as it serves only as a unique index and provides no predictive weight for the model bias reduction.

02

Imputation Strategy:

Handling missing alcohol_freq values by imputing with "Unknown" to avoid deleting critical historical records.

03

Initial Feature Reduction to Prevent Data Leakage:

Removing columns like claims_count and total_claims_paid as they occur after the prediction event. After removing these future-derived features that could cause data leakage, the remaining safe features were further evaluated to identify the top 10 most efficient predictors.

04

Model Bias Protection:

Isolating evaluation to ensure the cleaned features represent data available at the time of policy deployment.

Cleaning Results	
Features Dropped	1 (Index)
Missing Handled	30,083
Leakage Fixes	3 Columns
Status	Ready

Feature Engineering & Transformation

01

Categorical Encoding transforming 10 categorical (text) features into numerical formats so they can be mathematically processed by Machine Learning algorithms:

- **One-Hot Encoding:** Applied to non-hierarchical variables (e.g., Sex, Region, Marital Status) to prevent the model from assigning unintended weight or rank to specific categories.
- **Ordinal Encoding:** Applied to hierarchical/ranked variables (e.g., Education or Network Tier: Bronze to Platinum) to preserve their natural order.

02

Feature Scaling (Standardization):

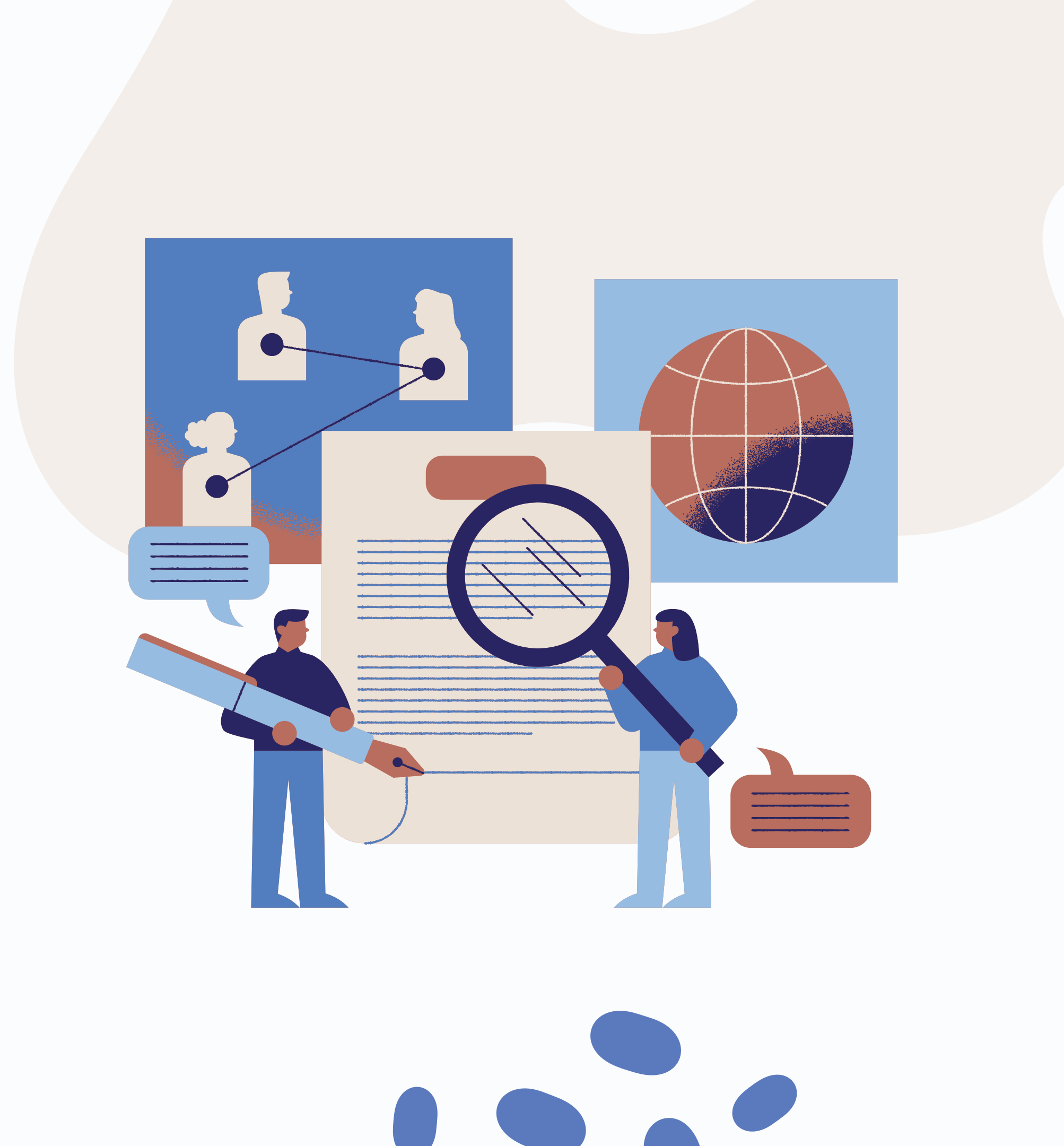
Standardizing the scale of all numerical features using StandardScaler. This is crucial to address massive disparities in value ranges (e.g., Income reaching hundreds of thousands vs. Age/BMI in tens).

03

Final Data Preparation:

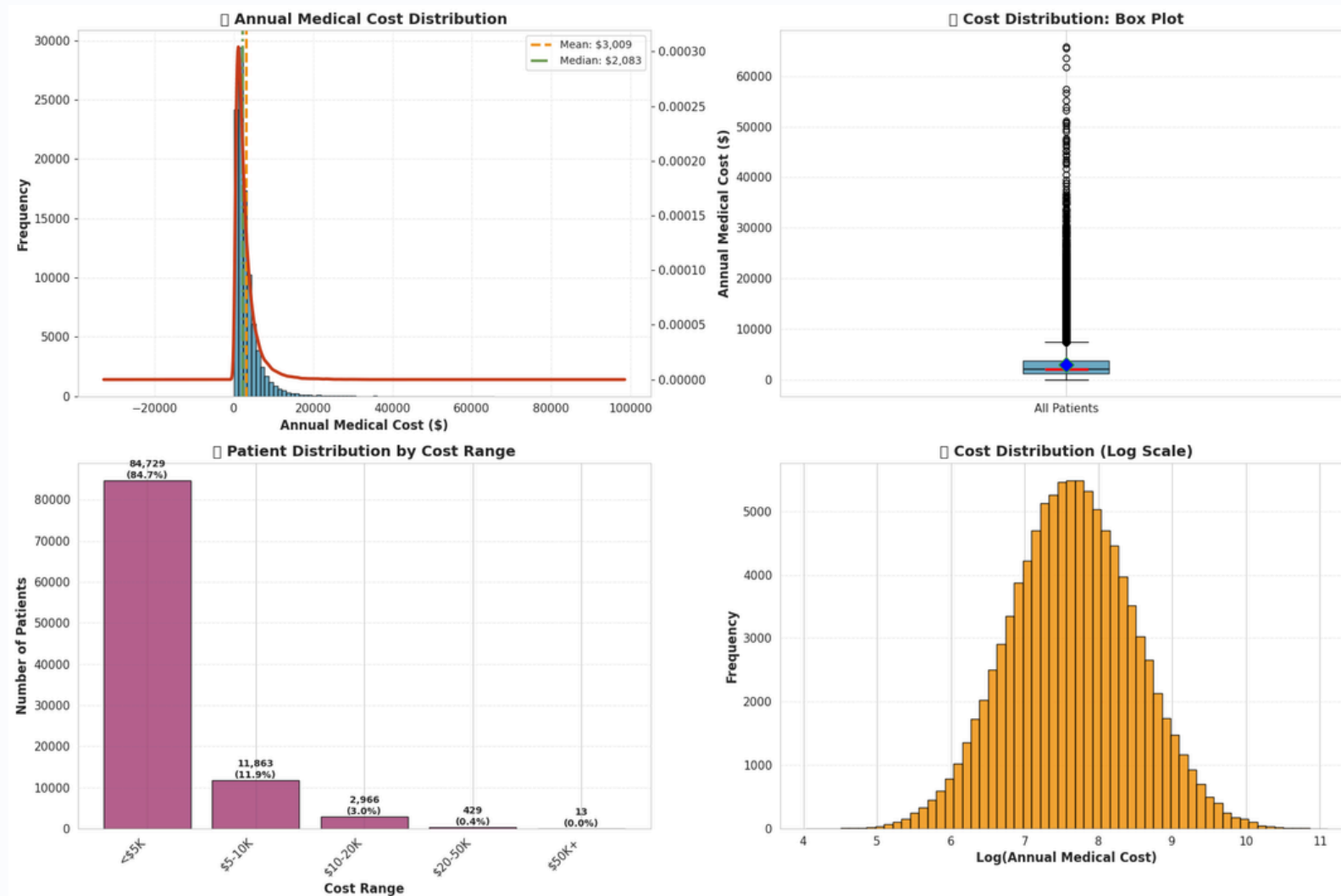
Ensuring all features are on a uniform scale so the model doesn't disproportionately favor features merely because of their large nominal values.

Key Insight



Key Insight

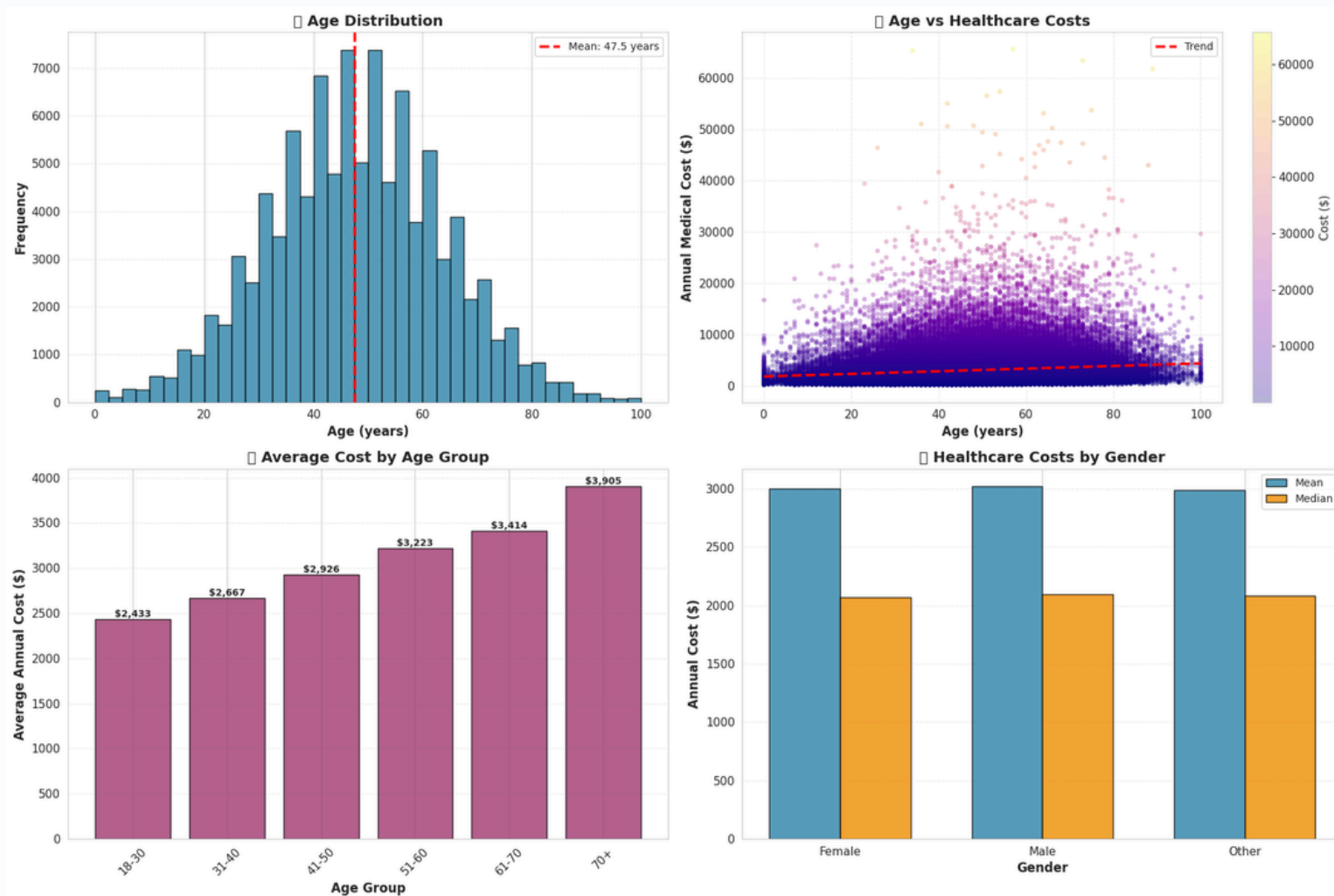
Annual Medical Cost Distribution



- 01 Average annual cost: \$3,009.45**
- 02 Median annual cost: \$2,082.57**
- 03 Healthcare costs are unevenly distributed (right-skewed) (skewness: 4.03)**
- 04 Approximately 84% of patients incur annual medical costs below \$5,000**
- 05 The wide variation in healthcare costs indicates a highly diverse patient population with significantly different healthcare needs and treatment requirements**

Key Insight

Age & Demographics

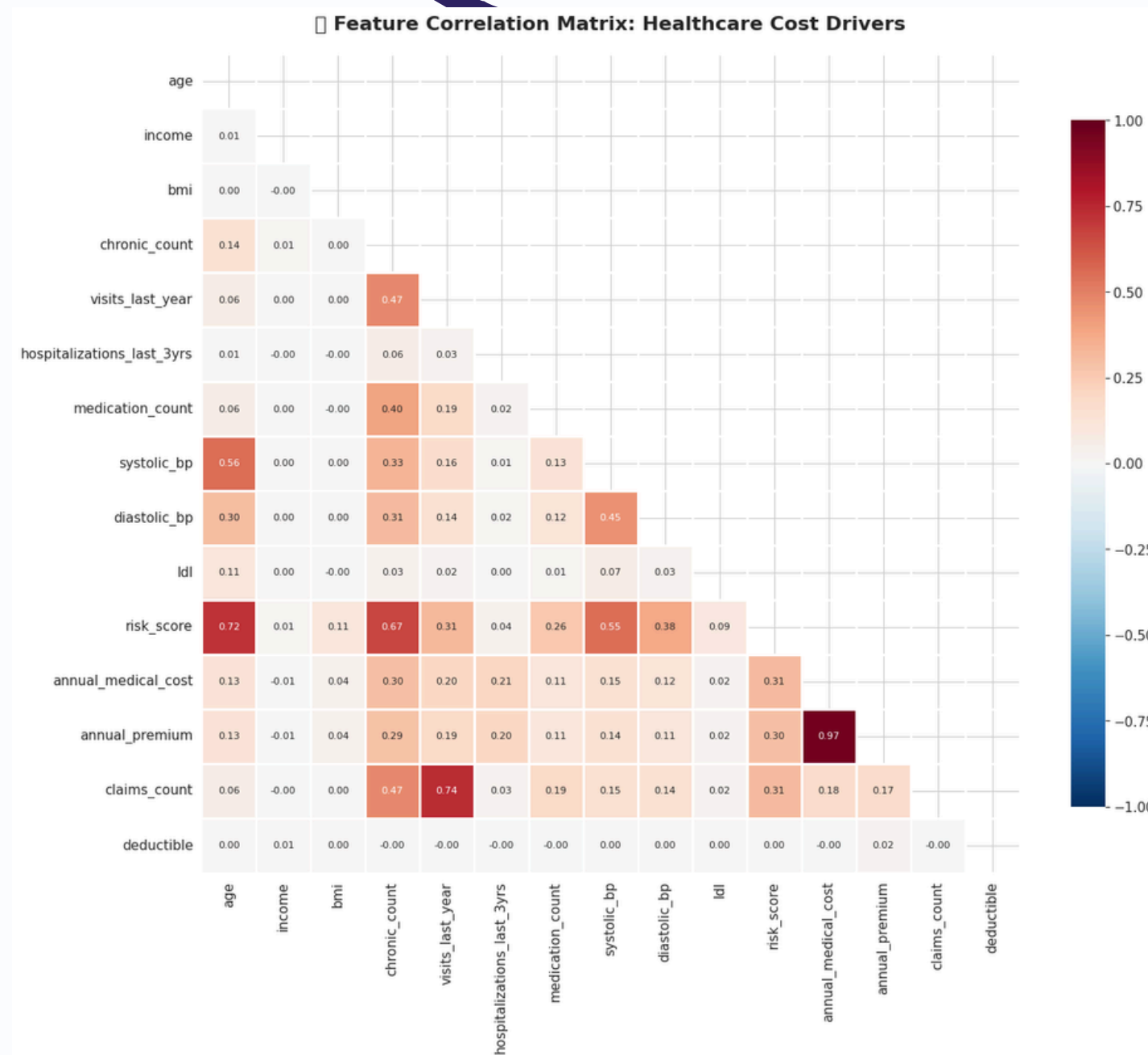


- 01 Mean age: 47.5 years
- 02 The majority of patients are between 40 and 60 years old
- 03 Healthcare costs generally increase with age
- 04 Average annual cost (Female): \$3,000.03
- 05 Average annual cost (Male): \$3,019.94
- 06 Age is a strong predictor of healthcare expenses

Key Insight

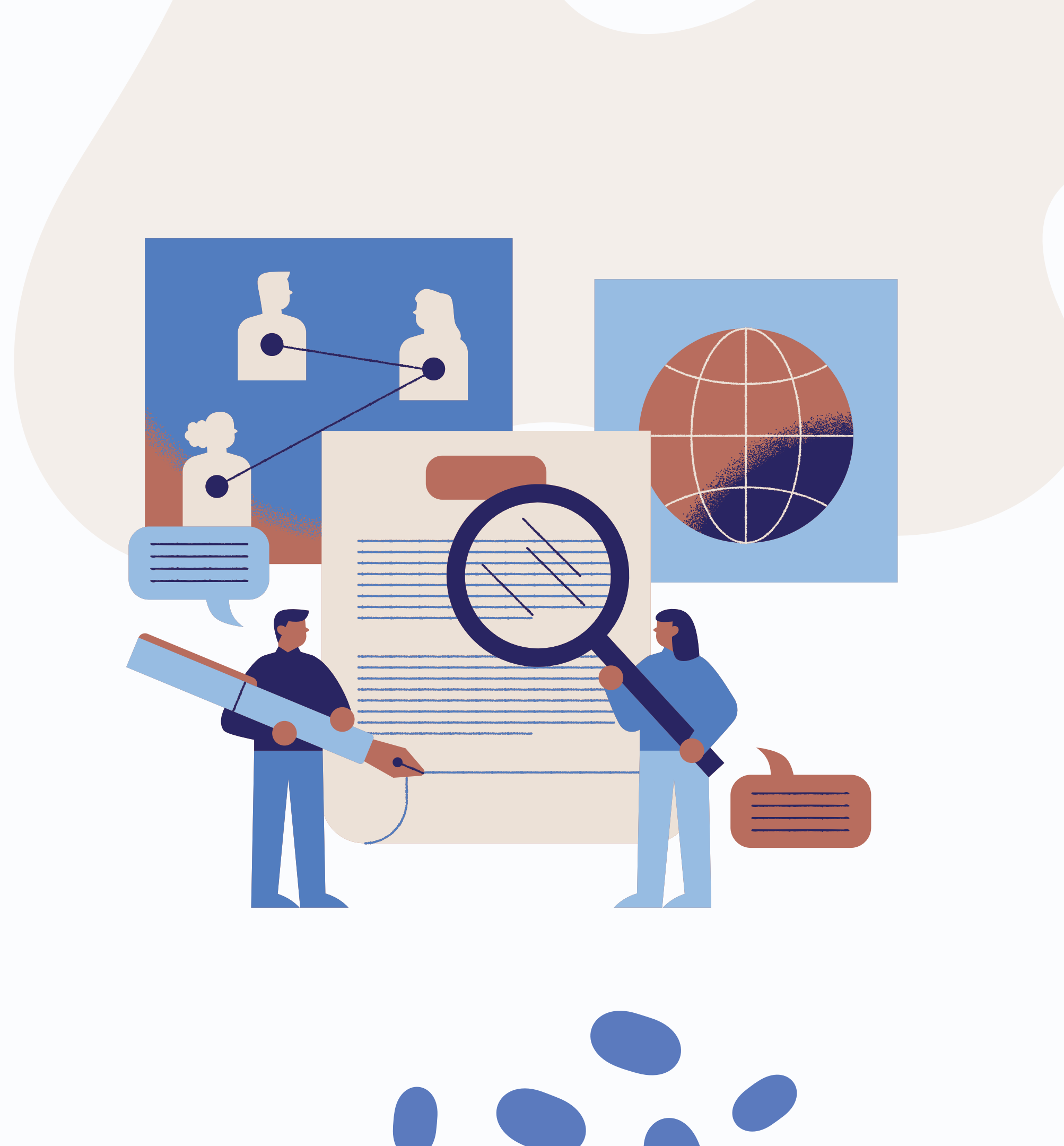
Feature Correlations with Medical Costs

- 01 The strongest correlation is between annual medical cost and annual insurance premium (0.97)**
- 02 Moderate positive correlations are observed with risk score ($r = 0.31$) and chronic condition count ($r = 0.30$)**
- 03 Factors such as deductible amount, income level, and BMI show little to no correlation with annual medical costs in this dataset ($r = 0.00$ – 0.04).**



Feature Engineering

Capture the complex relationships in medical costs.



Feature Engineering

```
# 1. HEALTHCARE UTILIZATION INTENSITY FEATURES
df_fe['visits_per_month'] = df_fe['visits_last_year'] / 12
df_fe['hosp_intensity'] = df_fe['hospitalizations_last_3yrs'] / 3 # Per year
df_fe['days_per_hospitalization'] = df_fe['days_hospitalized_last_3yrs'] / (df_fe['hospitalizations_last_3yrs'] + 1)

# Only create total_procedures if procedure columns exist
proc_cols = ['proc_imaging', 'proc_surgery', 'proc_pscho', 'proc_lab']
existing_proc_cols = [col for col in proc_cols if col in df_fe.columns]
if existing_proc_cols:
    df_fe['total_procedures'] = df_fe[existing_proc_cols].sum(axis=1)
else:
    df_fe['total_procedures'] = 0

df_fe['utilization_score'] = (df_fe['visits_last_year'] +
                             df_fe['hospitalizations_last_3yrs'] * 10 +
                             df_fe['medication_count'] * 2)
print("✓ Created healthcare utilization features")
```

1. Healthcare Utilization Features:

Higher healthcare utilization is generally associated with higher annual medical costs.

Feature Engineering

```
# 2. HEALTH RISK COMPOSITE SCORES
chronic_conditions = ['hypertension', 'diabetes', 'cardiovascular']
existing_chronic = [col for col in chronic_conditions if col in df_fe.columns]
if existing_chronic:
    df_fe['cardiovascular_risk'] = df_fe[existing_chronic].sum(axis=1).clip(upper=3)
else:
    df_fe['cardiovascular_risk'] = 0

df_fe['complex_patient'] = (df_fe['chronic_count'] >= 3).astype(int)
df_fe['high_risk_patient'] = (df_fe['risk_score'] > df_fe['risk_score'].quantile(0.75)).astype(int)

critical_conds = ['cancer_history', 'kidney_disease', 'liver_disease']
existing_critical = [col for col in critical_conds if col in df_fe.columns]
if existing_critical:
    df_fe['critical_conditions'] = df_fe[existing_critical].sum(axis=1)
else:
    df_fe['critical_conditions'] = 0

print("✓ Created health risk composite scores")

# 3. AGE-HEALTH INTERACTION FEATURES
df_fe['age_chronic_interaction'] = df_fe['age'] * df_fe['chronic_count']
df_fe['age_risk_score'] = df_fe['age'] * df_fe['risk_score']
df_fe['elderly_chronic'] = ((df_fe['age'] > 65) & (df_fe['chronic_count'] >= 2)).astype(int)
df_fe['young_high_risk'] = ((df_fe['age'] < 40) & (df_fe['chronic_count'] >= 1)).astype(int)
print("✓ Created age-health interaction features")
```

2. Health Risk Features

Patients with multiple chronic conditions tend to generate higher healthcare expenses.

3. Age & Health Interaction Features

Older patients with chronic diseases usually require more expensive healthcare treatment.

Feature Engineering

```
# 4. INCOME & INSURANCE FEATURES
df_fe['income_to_cost_ratio'] = df_fe['income'] / (df_fe['annual_medical_cost'] + 1)
df_fe['premium_burden'] = df_fe['annual_premium'] / (df_fe['income'] + 1)
df_fe['deductible_to_income'] = df_fe['deductible'] / (df_fe['income'] + 1)
df_fe['out_of_pocket_risk'] = df_fe['deductible'] + df_fe['copay']
df_fe['coverage_adequacy'] = df_fe['income'] / (df_fe['deductible'] + df_fe['copay'] + 1)
print("✓ Created income and insurance features")

# 5. LIFESTYLE RISK FACTORS
df_fe['bmi_category'] = pd.cut(df_fe['bmi'], bins=[0, 18.5, 25, 30, 100],
                              labels=['Underweight', 'Normal', 'Overweight', 'Obese'])
df_fe['is_obese'] = (df_fe['bmi'] >= 30).astype(int)
df_fe['is_overweight'] = (df_fe['bmi'] >= 25).astype(int)

# Lifestyle risk score - FIXED TYPE HANDLING
lifestyle_risk = df_fe['is_obese'].astype(int) * 2

if 'smoker' in df_fe.columns:
    # Convert smoker to numeric if it's not already
    if df_fe['smoker'].dtype == 'object':
        df_fe['smoker_numeric'] = df_fe['smoker'].map({'Yes': 1, 'No': 0, 1: 1, 0: 0}).fillna(0).astype(int)
    else:
        df_fe['smoker_numeric'] = df_fe['smoker'].astype(int)
    lifestyle_risk = lifestyle_risk + (df_fe['smoker_numeric'] * 3)

if 'exercise_frequency' in df_fe.columns:
    lifestyle_risk = lifestyle_risk + (7 - df_fe['exercise_frequency']).clip(lower=0)

df_fe['lifestyle_risk_score'] = lifestyle_risk
print("✓ Created lifestyle risk features")
```

4. Insurance & Financial Features

Insurance structure and financial capability can influence healthcare utilization behavior.

5. Lifestyle Risk Features

were added to help the model capture nonlinear relationships and reduce skewness in the data.

Feature Engineering

```
# 6. CLINICAL METRICS FEATURES
df_fe['bp_ratio'] = df_fe['systolic_bp'] / (df_fe['diastolic_bp'] + 1)
df_fe['pulse_pressure'] = df_fe['systolic_bp'] - df_fe['diastolic_bp']
df_fe['hypertensive'] = ((df_fe['systolic_bp'] > 140) | (df_fe['diastolic_bp'] > 90)).astype(int)
df_fe['high_cholesterol'] = (df_fe['ldl'] > 130).astype(int)

if 'hba1c' in df_fe.columns:
    df_fe['diabetic_control'] = (df_fe['hba1c'] > 6.5).astype(int)
print("✓ Created clinical metrics features")

# 7. UTILIZATION EFFICIENCY FEATURES
df_fe['cost_per_visit'] = df_fe['annual_medical_cost'] / (df_fe['visits_last_year'] + 1)
df_fe['cost_per_medication'] = df_fe['annual_medical_cost'] / (df_fe['medication_count'] + 1)
df_fe['avg_claim_size'] = df_fe['total_claims_paid'] / (df_fe['claims_count'] + 1)
df_fe['claims_frequency'] = df_fe['claims_count'] / 12 # Per month
print("✓ Created utilization efficiency features")
```

```
# 10. POLYNOMIAL FEATURES (for key predictors)
df_fe['age_squared'] = df_fe['age'] ** 2
df_fe['bmi_squared'] = df_fe['bmi'] ** 2
df_fe['chronic_squared'] = df_fe['chronic_count'] ** 2
df_fe['visits_squared'] = df_fe['visits_last_year'] ** 2
df_fe['income_squared'] = df_fe['income'] ** 2
print("✓ Created polynomial features")
```

6. Clinical Features

Abnormal clinical conditions often increase healthcare risk and medical spending.

7. Polynomial & Log Features

Added to help the model capture nonlinear relationships and reduce skewness in the data

Preparing Data

```
# Quick feature importance check using Random Forest
from sklearn.ensemble import RandomForestRegressor

# Prepare data (select only numeric features)
drop_cols = ['annual_medical_cost', 'log_medical_cost', 'person_id', 'bmi_category',
             'cost_range', 'age_group', 'hosp_category', 'med_category', 'deduct_category',
             'sex', 'region', 'urban_rural', 'education', 'marital_status', 'employment_status',
             'plan_type', 'network_tier']

X_temp = df_fe.drop(drop_cols, axis=1, errors='ignore')

# Keep only numeric features
X_temp = X_temp.select_dtypes(include=[np.number])

# Fill any NaN values
X_temp = X_temp.fillna(X_temp.mean())

y_temp = df_fe['annual_medical_cost']

# Quick RF for feature importance
print("Training Random Forest for feature importance analysis...")
rf_temp = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_depth=10)
rf_temp.fit(X_temp, y_temp)

# Get feature importances
feature_importance = pd.DataFrame({
    'feature': X_temp.columns,
    'importance': rf_temp.feature_importances_
}).sort_values('importance', ascending=False).head(25)
```

Feature importance analysis was performed using the **Random Forest Regressor model** to identify which variables had the greatest influence on annual medical cost prediction.

Random Forest was **chosen** because it can effectively capture nonlinear relationships and interactions between healthcare-related variables.

Preparing Data

```
# Select features (drop target and categorical columns)
drop_cols = ['annual_medical_cost', 'log_medical_cost', 'person_id', 'bmi_category',
             'cost_range', 'age_group', 'hosp_category', 'med_category', 'deduct_category',
             'sex', 'region', 'urban_rural', 'education', 'marital_status', 'employment_status',
             'plan_type', 'network_tier', 'income_to_cost_ratio', 'cost_per_visit',
             'cost_per_medication', 'avg_claim_size'] # Remove target leakage features

X = df_fe.drop(drop_cols, axis=1, errors='ignore')

# Keep only numeric features
X = X.select_dtypes(include=[np.number])

# Fill any NaN values
X = X.fillna(X.mean())

# Target variable
y = df_fe['annual_medical_cost']

print(f"✓ Features shape: {X.shape}")
print(f"✓ Target shape: {y.shape}")
print(f"✓ Number of features: {X.shape[1]}")

# Train-Test Split (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"\n✓ Training set: {X_train.shape[0]:,} samples")
print(f"✓ Test set: {X_test.shape[0]:,} samples")
print(f"✓ Train mean cost: ${y_train.mean():,.2f}")
print(f"✓ Test mean cost: ${y_test.mean():,.2f}")
```

This stage was performed to prepare the dataset before training the machine learning models.

- Unnecessary columns, categorical features, and leakage features were removed.
- Only numeric features were selected for modeling.
- Missing values were handled using mean imputation.
- The dataset was separated into:
 - **X** → *input features*
 - **y** → *target variable* (annual_medical_cost)

The data was then split into:

- 80% training data
- 20% testing data

**// This step ensures the model can learn from training data //
and be evaluated fairly on unseen data.**

Preparing Data

Data Preparation Results

```
=====
🍌 PREPARING DATA FOR MODELING
=====
✓ Features shape: (100000, 105)
✓ Target shape: (100000,)
✓ Number of features: 105

✓ Training set: 80,000 samples
✓ Test set: 20,000 samples
✓ Train mean cost: $3,004.87
✓ Test mean cost: $3,027.79

✓ Features scaled using StandardScaler
=====
```

// This indicates that the train and test data //
distributions were balanced and consistent.

The final dataset contained:

100,000 rows of patient data
105 features used for modeling

The data was divided into:

80,000 samples for training
20,000 samples for testing

The average annual medical cost:

\$3,004.87 in the training set
\$3,027.79 in the testing set

Result



MODEL COMPARISON RESULTS

Model	R ² Score	RMSE	MAE	MAPE (%)
Random Forest	0.95504	665.141032	360.597699	12.794538
Gradient Boosting	0.954222	671.163499	363.056535	12.906753
Ridge	0.948941	708.822128	416.056752	17.953664
Linear Regression	0.948937	708.847228	416.16541	17.966195
Lasso	0.94878	709.939293	410.382427	16.86695
Decision Tree	0.940189	767.167742	383.5546	13.492742
LightGBM	0.938175	779.976562	373.553701	13.163506
XGBoost	0.936693	789.27179	371.378957	12.931253
AdaBoost	0.917904	898.798018	642.372888	46.250453
KNN	0.793793	1424.46572	950.957515	50.518359
ElasticNet	0.686361	1756.774303	1045.278476	62.10824

Final Conclusion

This project successfully developed a healthcare cost prediction system using 100,000 patient records. Among 11 regression models, Random Forest delivered the best performance ($R^2 = 95.5\%$, MAPE = 12.8%).

The results show that age, risk score, chronic conditions, and insurance premiums are key drivers of healthcare costs. The final model was deployed as a web application, enabling real-time healthcare cost estimation.

UI DEPLOY

 **Medical Cost Prediction**
Predict annual medical costs based on patient health and insurance information.

 **Patient Information**

Age	Medication Count
30 - +	1 - +
Income	Risk Score
50000.00 - +	0.30
BMI	Annual Premium
25.00 - +	2000.00 - +
Chronic Disease Count	Deductible
0 - +	1000.00 - +
Visits Last Year	Copay
2 - +	20.00 - +

Predict Medical Cost

 **Prediction Result**

Estimated Annual Medical Cost: \$5,215.29

Deployment

The model was deployed as a simple web application where users can enter their health and insurance information to receive an estimated annual medical cost.

After the user clicks "Predict Medical Cost", the system uses the trained Random Forest model to generate a prediction instantly.

This deployment makes the model easy to use and demonstrates its real-world application for healthcare cost estimation.

THANKYOU